GRPO with QLoRA for Verified CISC-to-RISC Assembly Transpilation

Mid-point Report

Enzo Shiraishi Nunes da Silva
Mohammed Bin Zayed University of Artificial Intelligence
enzo.shiraishi@mbzuai.ac.ae
PhD in Computer Science

Sergei Pasynkov
Mohammed Bin Zayed University of Artificial Intelligence
sergei.pasynkov@mbzuai.ac.ae
PhD in Computer Science

Zayed Khaled Al Tamimi
Mohammed Bin Zayed University of Artificial Intelligence
zayed.altamimi@mbzuai.ac.ae
PhD in Computer Science

1 Introduction

Translating assembly from one processor family to another, for example from x86 to ARM, is a practical need when software must move to new hardware. It is difficult to do by hand and difficult to guarantee. Guaranteed Guess (GG) addresses this with a language model that proposes a translation and software tests that accept or reject it [1]. In this report, we study whether the same tests can also be used to train the model, with reinforcement learning, so that it produces more translations that pass.

Our earlier proposal applied DAPO to vision-language-action (VLA) models, which produce robot actions from images and instructions [9]. We changed the topic for two reasons. First, research on VLAs is currently centered on deployment cost, namely latency and memory [4, 5]. The existing reinforcement learning work for VLAs reports task success and relies on robot simulation to produce each training sample, and the feedback from these simulations is often sparse [6–8]. This is costly under limited compute. Second, translating assembly poses the same research question in a setting that our compute can support. The output can be checked exactly by running tests, one attempt is a single generation followed by a test run, and the model is small enough for parameter-efficient training on one GPU [1]. The problem is also practically relevant, because the translated code of this approach runs faster than Apple's Rosetta 2 [1].

The method we study is Group Relative Policy Optimization (GRPO) with a test-based reward, combined with Quantized Low-Rank Adaptation (QLoRA) to reduce memory use [11, 21]. Supervised fine-tuning is a necessary first stage for GRPO. GG released the model that results from this stage, so we start from that checkpoint instead of repeating that stage, which saves compute without changing the pipeline.

The rest of this report contains a short review of related work (Section 2), the baseline we evaluate (Section 3), the progress and plan for our method (Section 4), and the remaining work with its distribution among the authors (Section 5).

Preprint.

2 Related Work

Reinforcement learning with verifiable rewards. Reinforcement learning with verifiable rewards (RLVR) uses a program to check the answer instead of a learned reward model [10]. GRPO suits this setting because it compares several attempts at the same problem and does not require a separate model to predict their results [11]. The approach has given strong results on verifiable tasks such as mathematics and coding [12], and GRPO with binary rewards has been shown to increase the success probability of the starting model [13].

RLVR for code. Code is a natural fit for RLVR because tests give an exact and inexpensive result. RLEF trains code models on the results of running their own code [14], and VeRPO uses partial test results as a denser reward [15]. Recent works apply the same idea to low-level code, where the output can be executed or formally verified. SuperCoder uses reinforcement learning to produce faster assembly programs, rewarding both correctness and speed [16]. LLM-VeriOpt trains a small model with GRPO, using a formal verifier as the reward, to optimize compiler code [17]. AutoDecompiler uses compilation and test feedback to train a model that converts binaries back into source code [18], and TRAVEL applies reinforcement learning with execution feedback to translate C code into Rust [19]. These works show that RLVR can improve code, assembly, and translation. To our knowledge, none of them applies it to translating CISC assembly into RISC assembly.

Assembly translation. Guess & Sketch combines guesses from a language model with a solver that checks whether the translation is equivalent [3]. CRT is a lightweight language model that translates x86 assembly into ARM and RISC-V [2]. GG fine-tunes small pre-trained code models (DeepSeek-Coder 1.3B and Qwen2.5-Coder 0.5B and 1.5B) on about 1.3 million C programs from AnghaBench and The Stack v2, compiled to both instruction sets. It generates candidate translations and checks them with software tests. It is evaluated on HumanEval, a set of programming problems whose C versions are translated and then executed, and on BringupBench, a set of larger programs for embedded systems [1]. In GG, the tests only accept or reject the output of the model. We use the same tests as a training signal.

Parameter-efficient training. Low-Rank Adaptation (LoRA) trains a small number of added weights instead of the whole model [20]. QLoRA does the same on top of a base model stored in 4-bit precision, which further reduces memory use [21]. We use both so that training fits on a single GPU.

3 Baseline

Our baseline is the released GG model for ARMv8 at optimization level O2, which is Qwen2.5-Coder-1.5B-Instruct fine-tuned on the GG data [1]. We do not train it. We evaluate it with our own reward harness: the predicted ARM assembly is the only definition of the function in a test program, which is assembled, linked, and run. An empty or truncated prediction therefore fails. The test programs are the ones released by the GG authors with their data.

We evaluate on the 164 HumanEval-C problems at O2 and report the pass rate, that is, the fraction of problems whose translation passes all tests. We report it under two decoding settings: beam search, as in the original evaluation, and sampling at temperature 1, which matches how the model is used during GRPO. The first setting also checks whether our harness reproduces the number reported for this model [1].

Table 1 gives the results. As a check of the harness, the reference ARM assembly passes all 164 problems, and an empty prediction fails all 164. With beam search, the model solves 72 problems in our first run and 77 in a second run that uses the software versions listed with the released model, while the authors report 74. The two runs use the same decoding, and none of the problems that change between them reach the length limit, so the difference comes from the software versions alone. These versions can therefore move the result by about five problems, or 3 percentage points. For this reason, we evaluate the baseline and our trained models with the same software versions. We also found that the authors' released script turns on sampling during beam search, through the configuration file of the model, although the paper states that sampling was disabled. A run of that script is in progress.

Table 1: Problems solved by the released GG model at O2. BringUp results are out of the 65 programs that can be built, which matches the percentages in the GG paper. The BringUp runs use PyTorch 2.7 instead of 2.6, because our GPU requires it.

Decoding and software	HumanEval-C	BringUp
Reported by GG [1], beam search	74/164 (45.1%)	10/65 (15.4%)
Beam search, newer versions, 16,000 new tokens	72/164 (43.9%)	–
Beam search, versions listed with the model	77/164 (47.0%)	[in progress]
Beam search, versions used for training	[in progress]	–
Authors' scripts as released	[in progress]	[in progress]
Sampling at temperature 1	[pending]	–

4 Proposed Method

Method. We apply GRPO to the GG checkpoint. For each problem, the model generates a group of eight translations. Each translation receives a reward of 1 if it passes all tests and 0 otherwise. The model is updated to favor the translations that pass relative to the rest of the group. We use the GRPO implementation in the TRL library, and we write the reward function, the data preparation, and the evaluation code ourselves. The two training variants differ only in the base weights: LoRA on a 16-bit base model, and QLoRA on a 4-bit base model. We compare both with the baseline of Section 3. The training problems come from MBPP-C, which has assertion tests. Of its 973 problems that compile, we keep the 952 whose reference ARM assembly passes its own tests. We will add ExeBench, which has real input and output examples, only if more training problems are needed. HumanEval-C is reserved for evaluation. Both variants use identical settings and are trained for 60 steps, one pass over the training problems.

Settings. Each step uses 16 problems with eight translations each. A translation can have at most 4,096 tokens, which covers the reference translations of all but 11 training problems, and translations cut at this limit are not used in the update. The LoRA weights have rank 1 and are added to all linear layers, following results that this rank is enough for reinforcement learning [22]. The learning rate is 5×10^{-5}. We use the loss of DAPO, which averages over all tokens of the batch [9], and no penalty for moving away from the starting model. The 4-bit base model of QLoRA uses the NF4 format with double quantization, as in the QLoRA paper [21].

Progress. The reward harness described above has been implemented. It runs on a Mac, because the GG data is ARM assembly produced by Apple's compiler and can only be assembled and run natively on Apple Silicon. The training machine is a Linux server with one NVIDIA RTX PRO 6000 Blackwell GPU with 96 GB, which is the compute available to us, and it reaches the harness through a single network tunnel. This differs from our earlier assumption of up to two GPUs. We also prepared the code that loads the problems and the tests.

Before training, we had to match the toolchain. The official GG data was produced with Xcode 14.3 and the macOS 13.3 SDK, while our Mac has a newer toolchain, and data regenerated with it differs from the official data. We therefore built a macOS 13 virtual machine with Xcode 14.3, regenerated our data in it, and run the harness inside it. We treated the work as complete when regenerated HumanEval-C output matched the official data. This step is complete: compiling the authors' C code in this machine reproduces all 328 official HumanEval-C files exactly.

A first short training run of the LoRA variant works from start to end.

Risks. The reward harness runs on a single Mac, so its throughput limits the number of training steps. We have not yet verified that updated LoRA weights are passed correctly to the fast generation engine when the base model is 4-bit. We will test this on a short run before any long training. We have not yet checked whether the training problems overlap with the evaluation problems or with the GG training data, and we will check this before training.

5 Remaining Work and Distribution

Dates	Work
Oct. 24–30	Complete the baseline runs of Table 1 and the short tests of both variants.
Oct. 31–Nov. 13	Train the LoRA and QLoRA variants.
Nov. 14–27	Evaluate both variants, add a third baseline, run one ablation.
Nov. 28–Dec. 7	Analyze results and failures, prepare the final report and code.

The final report will compare against at least three baselines: the released GG model, the LoRA variant, and a third chosen after the first results.

The team will divide the work between training the LoRA and QLoRA variants, evaluating them and the baselines, and analyzing the results for the final report.

References

[1] A. Heakl, S. Hashmi, C. Abi, C. Lee, and A. Mahmoud. Guaranteed Guess: A Language Modeling Approach for CISC-to-RISC Transpilation with Testing Guarantees. arXiv:2506.14606, 2025. https://arxiv.org/abs/2506.14606.

[2] A. Heakl, C. Abi, R. Hossam, and A. Mahmoud. From CISC to RISC: Language-Model Guided Assembly Transpilation. arXiv:2411.16341, 2024. https://arxiv.org/abs/2411.16341.

[3] C. Lee, A. Mahmoud, M. Kurek, S. Campanoni, D. Brooks, S. Chong, G.-Y. Wei, and A. M. Rush. Guess & Sketch: Language Model Guided Transpilation. arXiv:2309.14396, 2023. https://arxiv.org/abs/2309.14396.

[4] W. Guan, Q. Hu, A. Li, and J. Cheng. Efficient Vision-Language-Action Models for Embodied Manipulation: A Systematic Survey. arXiv:2510.17111, 2025. https://arxiv.org/abs/2510.17111.

[5] Z. Yu et al. A Survey on Efficient Vision-Language-Action Models. arXiv:2510.24795, 2025. https://arxiv.org/abs/2510.24795.

[6] H. Li et al. SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning. arXiv:2509.09674, 2025. https://arxiv.org/abs/2509.09674.

[7] G. Lu et al. VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning. arXiv:2505.18719, 2025. https://arxiv.org/abs/2505.18719.

[8] H. Zang et al. RLinf-VLA: A Unified and Efficient Framework for Reinforcement Learning of Vision-Language-Action Models. arXiv:2510.06710, 2025. https://arxiv.org/abs/2510.06710.

[9] Q. Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476, 2025. https://arxiv.org/abs/2503.14476.

[10] N. Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124, 2024. https://arxiv.org/abs/2411.15124.

[11] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024. https://arxiv.org/abs/2402.03300.

[12] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Nature, 645:633–638, 2025; arXiv:2501.12948. https://arxiv.org/abs/2501.12948.

[13] Y. Mroueh. Reinforcement Learning with Verifiable Rewards: GRPO's Effective Loss, Dynamics, and Success Amplification. arXiv:2503.06639, 2025. https://arxiv.org/abs/2503.06639.

[14] J. Gehring, K. Zheng, J. Copet, V. Mella, Q. Carbonneaux, T. Cohen, and G. Synnaeve. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. arXiv:2410.02089, 2024. https://arxiv.org/abs/2410.02089.

[15] L. Wang et al. Beyond Binary: Turning Partial Success into Dense Verifiable Rewards for Reinforcement Learning in Code Generation. arXiv:2601.03525, 2026. https://arxiv.org/abs/2601.03525.

[16] A. Wei, T. Suresh, H. Tan, Y. Xu, G. Singh, K. Wang, and A. Aiken. SuperCoder: Assembly Program Superoptimization with Large Language Models. arXiv:2505.11480, 2025. https://arxiv.org/abs/2505.11480.

[17] X. Fang, J. Kang, R. C. Rocha, S. Ainsworth, and L. Mukhanov. LLM-VeriOpt: Verification-Guided Reinforcement Learning for LLM-Based Compiler Optimization. IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2026. https://2026.cgo.org/details/cgo-2026-papers/37/LLM-VeriOpt-Verification-Guided-Reinforcement-Learning-for-LLM-Based-Compiler-Optimi.

[18] P. Liu et al. Binary Decompilation LLM with Feedback-Driven Multi-Turn Refinement. arXiv:2606.16162, 2026. https://arxiv.org/abs/2606.16162.

[19] F. Luo, J. Liu, C. Gao, J. Feng, and K. Liu. Towards Reliable C-to-Rust Translation with Rule-Guided Reasoning and Reinforcement Learning. arXiv:2607.19966, 2026. https://arxiv.org/abs/2607.19966.

[20] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR), 2022; arXiv:2106.09685. https://arxiv.org/abs/2106.09685.

[21] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. Advances in Neural Information Processing Systems (NeurIPS), 2023; arXiv:2305.14314. https://arxiv.org/abs/2305.14314.

[22] J. Schulman and Thinking Machines Lab. LoRA Without Regret. Thinking Machines Lab: Connectionism, 2025. https://thinkingmachines.ai/blog/lora/.